DATAKERN ASSIGNMENT

www.data-kern.com

DataKern Data Engineering — ADLS & Silver Layer Lab

From the Internet to a Silver Layer on ADLS and Databricks

> **Congratulations on your new job as a Data Engineer!** A city analytics team wants to know whether weather changes how New York moves. Nothing exists yet: no files, no tables. Your job is the first two layers of the lakehouse: **land the data in Azure Data Lake Storage, then build a Silver layer in Databricks that the team can trust.** Another engineer will build reports on top of your tables next week, so your names and numbers must be exactly right.

Why this assignment

You already know how to connect Databricks to ADLS and how to register an external table. This assignment makes you **use** that knowledge end to end, on real data with real problems, until it feels routine.

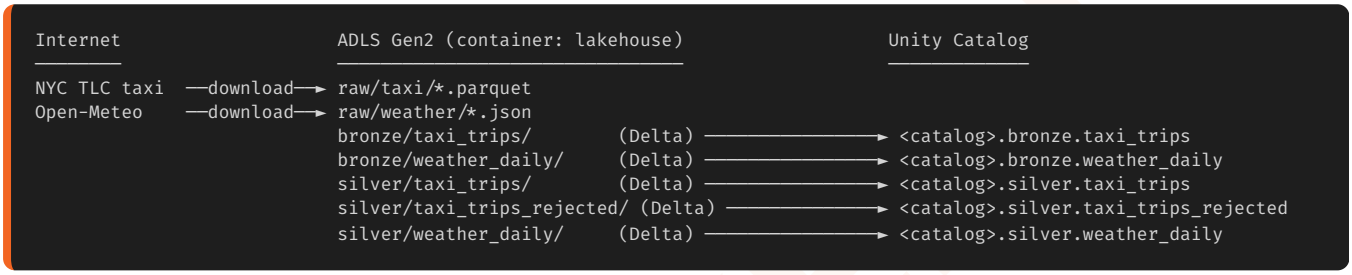
You will practise	Where
Downloading data and landing it in ADLS	Notebook 1, Step 1
Writing Delta files to a container and registering them as Unity Catalog external tables	Notebook 1, Steps 2–4 and Notebook 2, Step 5
Reading from Bronze and cleaning with PySpark	Notebook 2
Rejecting, fixing and keeping bad data deliberately	Notebook 2, Steps 2–3
Proving your pipeline is correct, not just "did not crash"	check cells and report §8

> **No deployment, no pipelines, no scheduling.** Two notebooks, run by hand. The goal is fluency with Azure and Databricks.

Important learning rule

> **AI is allowed. Understanding is not optional.** You may use any AI tool to help write code. You must be able to explain every line you submit, and you will **prove it** in `OBSERVATION_REPORT.md` (§11 explain-it-back, §12 how you used AI). If you cannot explain a line, it is not yours yet.
>
> Do not trust AI output blindly: check it against the **naming contract** and the **check cells**. AI does not know your container name.

What you will build



The naming contract

Every name below is **fixed**. The check cells and your instructor look for exactly these.

Item	Value
Storage	the ADLS Gen2 account you set up earlier
Container	`lakehouse` (one container, all layers)
Catalog	the **existing** catalog given to you (`SHOW CATALOGS`). You cannot create your own
Schemas	`bronze` , `silver` (lowercase, you create both)
Tables	external Delta tables only, always pointing at the same container
Months	`2025-08` , `2025-09` , `2025-10` (about 12.25 million taxi trips)

Table	Path inside the container	Source / content
`bronze.taxi_trips`	`bronze/taxi_trips/`	Taxi trips as published, plus `source_file` , `source_month` , `ingested_at` . Partitioned by `source_month`
`bronze.weather_daily`	`bronze/weather_daily/`	Daily NYC weather, one row per date, as published by the API
`silver.taxi_trips`	`silver/taxi_trips/`	Cleaned, typed, snake_case trips
`silver.taxi_trips_rejected`	`silver/taxi_trips_rejected/`	Rows that failed a rule, with `reject_reason`

Table	Path inside the container	Source / content
`silver.weather_daily`	`silver/weather_daily/`	Cleaned daily weather

Raw files: `raw/taxi/yellow_tripdata_2025-08.parquet` (also `-09`, `-10`) and `raw/weather/nyc_daily_2025-08-01_2025-10-31.json`.

Setup

1. Import both notebooks from this `starter/` folder into your Databricks workspace (**Workspace** → **Import**, or a Git folder).
2. Attach **serverless** compute.
3. In each notebook fill in the two values marked `< ... >` at the top: `STORAGE_ACCOUNT` and `CATALOG`.
4. Confirm your Databricks access to the container works (you did this earlier) before you start.
5. Copy `OBSERVATION_REPORT.md` into your own repo and fill it in **as you go**, not at the end.

Notebook	Job	Rough time
`01_ingest_bronze.ipynb`	Internet → `raw/` → Bronze Delta → Bronze external tables	1.5 h
`02_build_silver.ipynb`	Profile → reject → clean → Silver Delta → Silver external tables	2.5 h
`OBSERVATION_REPORT.md`	Evidence and explanations	1 h

How the notebooks work

Each step has: **what to build**, **rules**, **hints** (never the full answer), and a **Pause and observe** question. Write your code in the empty cell under each step.

You write the code for Steps 1–4 of both notebooks, plus the registration and the join query in Notebook 2, Step 5. The **check cell** at the end of each notebook is given: it tells you what is still wrong, with ✅ or ❌.

> **Validation rule:** "It ran without errors" is not done. Done means every check is ✅ **and** your report numbers agree with your own output.
>
> **Git checkpoints:** commit after each notebook, with a message that says what is now true (`"notebook 1: bronze ingestion, 10/10 checks"`).

What to submit

#	Deliverable
1	`01_ingest_bronze.ipynb` with outputs visible, all checks ✅
2	`02_build_silver.ipynb` with outputs visible, all checks ✅

#	Deliverable
3	`OBSERVATION_REPORT.md`, filled in completely, with the 3 screenshots in `evidence/`

How this is reviewed

Looks at	What the instructor wants to see
Names	Container, paths, schemas and tables exactly as in the contract
Numbers	Row counts match the instructor's (they should be identical for everyone) and reconcile in §8
Understanding	§6 decisions, §11 and §12 are in your own words and specific, not generic
Honesty	§10 problems and §12 AI corrections are real, not invented

Don't focus on getting the answer. Focus on learning how to engineer the solution. **— Uma Kiran | DataKern | www.data-kern.com**

DATAKERN